Graph Exploration DFS

Contents

Saltar al contenido principal

15.1. Introducción

Al trabajar con grafos, la búsqueda en profundidad (DFS) es un algoritmo importante para explorar todos los vértices y aristas. En este cuaderno, implementaremos DFS usando NumPy y NetworkX para encontrar un camino entre nodos en un grafo no ponderado y no dirigido.

15.2. ¿Cómo funciona DFS?

DFS explora un grafo profundizando lo máximo posible a lo largo de cada rama antes de retroceder:

1. Comenzamos en el nodo fuente y lo marcamos como visitado
2. Exploramos a uno de sus vecinos por completo (incluyendo a todos los

Saltar al contenido principal

vecinos de ese vecino) antes de pasar al siguiente vecino

3. Cuando no podemos profundizar más, retrocedemos hasta el último nodo con vecinos inexplorados
4. Este proceso continúa hasta que alcanzamos el nodo objetivo o visitemos todos los nodos accesibles

A diferencia de BFS, DFS puede no encontrar el camino más corto en grafos sin ponderación, pero a menudo es más eficiente en memoria y adecuado para ciertos problemas como la detección de ciclos y la ordenación topológica.

15.3. Implementación con NumPy

Recuerda que está prohibido usar funciones o librerías integradas que

Saltar al contenido principal

el algoritmo desde cero usando NumPy y listas estándar de Python. No son necesarias colas ni pilas para esta implementación, ya que podemos usar arrays NumPy para almacenar los nodos visitados y seguir el proceso de exploración. Cualquier implementación que utilice funciones DFS integradas o estructuras de datos de cola o pila será considerada inválida.

```
def dfs_numpy(graph, source, target
    """
    Implementation of DFS using Num

    Parameters:
    - graph: NetworkX graph (undire
    - source: Source node
    - target: Target node

    Returns:
    - steps: Number of steps needed
    - path: List of nodes in the pa
```

Saltar al contenido principal

```
    return steps, path
```

15.4. Ejemplo visual

Vamos a crear un grafo no dirigido sencillo para visualizar cómo funciona el algoritmo:

```
def create_example_graph():
    """Creates an example undirecte
    G = nx.Graph()

    # Add nodes (cities)
    cities = ['Madrid', 'Barcelona'
    G.add_nodes_from(cities)

    # Add edges (no weights)
    edges = [
        ('Madrid', 'Barcelona'),
        ('Madrid', 'Valencia'),
        ('Madrid', 'Sevilla'),
        ('Madrid', 'Bilbao'),
        ('Madrid', 'Zaragoza'),
```

Saltar al contenido principal

```
            ('Valencia', 'Sevilla'),
            ('Zaragoza', 'Bilbao')
        ]

        # Add the edges
        G.add_edges_from(edges)

        return G

# Create and visualize the graph
G = create_example_graph()

# Visualize the graph
pos = nx.spring_layout(G, seed=42)
plt.figure(figsize=(10, 8))
nx.draw(G, pos, with_labels=True, n
plt.title("Undirected Graph of Span
plt.show()
```

15.5. Probando nuestro algoritmo DFS

Vamos a ejecutar nuestro algoritmo en este gráfico y ver los resultados:

Saltar al contenido principal

```
# Define source and target nodes
source = 'Bilbao'
target = 'Sevilla'

# Run DFS
dfs_steps, dfs_path = dfs_numpy(G,
print(f"DFS Path from {source} to {
print(f"Number of steps: {dfs_steps
print(f"Path found: {dfs_path}")
print(f"Path length: {len(dfs_path)

# Compare with NetworkX (shortest p
shortest_path = nx.shortest_path(G,
print(f"\nNetworkX result (shortest
print(f"Shortest path: {shortest_pa
print(f"Path length: {len(shortest_

# Check if DFS found the shortest p
if len(dfs_path) == len(shortest_pa
    print("\nDFS found the shortest
else:
    print(f"\nDFS found a valid pat
```

15.6. Visualizing the DFS

Saltar al contenido principal

Process

Let's visualize the path that DFS finds:

```
def visualize_path(G, path, title):
    """Visualizes the path with num
    plt.figure(figsize=(10, 8))

    # Create a copy of the graph fo
    H = G.copy()
    pos = nx.spring_layout(H, seed=

    # Create the edges of the path
    edge_colors = []
    edge_widths = []

    # Create edge labels dictionary
    edge_labels = {}

    # Process all edges and mark th
    for u, v in H.edges():
        if len(path) > 1:
            # Check if this edge is
            is_path_edge = False
            for i in range(len(path
                if (u == path[i] an
```

```
                    # Add label wit
                    edge_labels[(u,
                    break

            if is_path_edge:
                edge_colors.append(
                edge_widths.append(
            else:
                edge_colors.append(
                edge_widths.append(
        else:
            edge_colors.append('gra
            edge_widths.append(1.0)

    # Draw all nodes
    nx.draw_networkx_nodes(H, pos,

    # Highlight the nodes in the pa
    nx.draw_networkx_nodes(H, pos,

    # Draw all edges with appropria
    nx.draw_networkx_edges(H, pos,

    # Draw edge labels (numbers) fo
    nx.draw_networkx_edge_labels(H,
                                 fon

    # Draw node labels
```

Saltar al contenido principal

```
    plt.title(title)
    plt.axis('off')
    plt.show()

# Visualize the DFS path with numbe
visualize_path(G, dfs_path, "DFS Pa

# Also visualize the shortest path
visualize_path(G, shortest_path, "S
```

15.7. Comparing DFS and BFS

Let's highlight the key differences between DFS and BFS for graph traversal:

```
# Run both algorithms on the same s
source = 'Bilbao'
target = 'Sevilla'

# For this example, we'll assume we
dfs_steps, dfs_path = dfs_numpy(G,
# Let's assume we also have the BFS
```

Saltar al contenido principal

```
print(f"Comparison of DFS and BFS f
print(f"DFS steps: {dfs_steps}, pat
print(f"BFS steps: {bfs_steps}, pat

print("\nDFS Path:")
print(" -> ".join(dfs_path))

print("\nBFS Path (Shortest Path):"
print(" -> ".join(bfs_path))
```

15.8. Conclusion

In this notebook, we have explored the Depth-First Search algorithm implemented using NumPy and NetworkX. DFS is a fundamental graph traversal

along each branch before backtracking. Unlike BFS, DFS may not find the shortest path in unweighted graphs, but it has several advantages for certain

Saltar al contenido principal

1. DFS generally requires less memory than BFS, as it doesn't need to store all nodes at a given depth level
2. DFS is well-suited for problems like topological sorting, finding connected components, and cycle detection
3. DFS can be implemented recursively (although we used an iterative approach here)

Our implementation leverages NumPy's efficient array operations and NetworkX's graph representation capabilities, demonstrating how these tools can be combined to solve graph-related problems effectively. The visualization of the path provides an intuitive understanding of how DFS works and how it differs from BFS. This algorithm is essential in many applications, including maze generation

and solving, web crawling with specific depth limits, and searching in game trees.

15.9. Exercise

15.9.1. Exercise 1

After we implemented the DFS algorithm using NumPy, now it's time to understand the algorithm. What we want you to do is to explain the algorithm in your own words, and try to execute each step of the algorithm, showing print statements for each step. This will help you understand the algorithm better and see how it works in practice.

15.9.2. Exercise 2

Apply the DFS algorithm to the synthetic graphs that you generated in the previous notebook, and discuss the results. How

Saltar al contenido principal

graphs? How does the DFS path compare to the shortest path found by BFS? In what cases might DFS be more appropriate than BFS?

Previous
14. Graph Exploration BFS

Next
16. Graph Exploration Random Walks